

VDC Teacher Mobile Application

Technical Requirements Document (TRD)

1. Purpose

This document defines the technical architecture, technology stack, database requirements, APIs, security, deployment, coding standards, integrations, and non-functional requirements for the VDC Teacher Mobile Application.

2. System Architecture

Architecture: React Native Android APK → Supabase (Authentication, PostgreSQL Database, Storage, Row Level Security) → Fast2SMS API.

The mobile application communicates directly with Supabase using secure HTTPS connections. No custom backend server is required for Version 1. Business logic is implemented using PostgreSQL functions, Row Level Security policies, and client-side services. Future versions can introduce a Spring Boot backend without changing the database.

3. Technology Stack

Frontend:

- React Native (Expo)
- TypeScript
- React Navigation
- NativeWind
- Zustand
- TanStack Query
- Axios (external APIs only)
- Expo FileSystem
- React Native Chart Kit

Backend Platform:

- Supabase Auth
- PostgreSQL
- Supabase Storage
- Supabase Edge Functions (optional for future automation)

External Integration:

- Fast2SMS API (DLT Approved)

Version Control:

- GitHub

AI Development:

- Claude Code / Cursor
- ChatGPT

4. Database Design

Primary tables:

Teacher, Student, Subject, Class, Teacher_Class, Attendance, Test, Test_Mark, Assignment, Assignment_Mark, Notes, SMS_Log.

Relationships:

Teacher 1..N Teacher_Class

Class 1..N Students

Class 1..N Attendance

Class 1..N Tests
Test 1..N Test_Mark
Assignment 1..N Assignment_Mark

Indexes:

Attendance(class_id,date)
Student(usn)
Teacher(email)
Test(class_id)
Assignment(class_id)

Constraints:

- One attendance record per student per class per day.
- Marks cannot exceed maximum marks.
- Foreign key integrity enabled.

5. Authentication & Authorization

Authentication uses Supabase Auth with email/password.

JWT tokens secure API access.

Role:

Teacher

Teachers can access only their assigned classes using PostgreSQL Row Level Security (RLS).

Passwords are never stored in plaintext.

6. Functional Modules

Authentication
Dashboard
Class Management
Attendance
Test Marks
Assignment Marks
Reports
Notes Repository
SMS Notifications

Attendance workflow:

Login → Dashboard → Select Class → Student List → Mark Attendance → Save → SMS sent to absentees → Reports updated.

7. API & Integration Requirements

Supabase client SDK will perform CRUD operations.

Fast2SMS API:

Trigger after attendance save.

Payload contains approved DLT template variables.

Delivery status logged in SMS_Log table.

Future APIs:

Push Notifications
Admin Portal
Student Portal.

8. Storage

Teacher notes (PDF, DOCX, PPTX, Images) stored in Supabase Storage.
Metadata stored in Notes table.
Download links generated securely.

9. Reporting

Monthly Attendance
Quarterly Attendance
Student-wise Attendance
Test Performance
Assignment Performance

Reports generated using SQL views and aggregate queries.
PDF export supported.

10. Offline Strategy

Attendance is cached locally when offline.
Automatic synchronization occurs when connectivity returns.
Conflict resolution uses latest valid update timestamp.

11. Security

HTTPS only.
JWT authentication.
RLS enabled.
Parameterized queries.
Secure file URLs.
Audit timestamps.
Application logs without sensitive information.

12. Coding Standards

TypeScript strict mode.
Component-based architecture.
Repository/service separation.
Reusable UI components.
ESLint + Prettier.
Git feature branching.

13. Deployment

Development:
Expo Go

Testing:
Internal APK

Production:
Android APK

Cloud:
Supabase project with production database, storage and authentication.

14. Performance Requirements

Login <2 sec
Attendance save <3 sec
Reports <5 sec

Support at least 100 concurrent users
99.9% availability target.

15. Future Technical Enhancements

Admin Web Portal
Spring Boot microservices
Parent & Student apps
AI analytics
OCR attendance
Voice attendance
Push notifications
CI/CD pipeline
Automated backups
Monitoring & alerting